

```
In [1]: # Project 9 - Combined Random Forest & Neural Network Classifiers
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split, KFold, cross_val_score
from sklearn.preprocessing import StandardScaler
from sklearn.ensemble import RandomForestClassifier
from sklearn.neural_network import MLPClassifier
from sklearn.metrics import confusion_matrix, roc_auc_score, accuracy_score, recall
import warnings

sns.set_style("whitegrid")
warnings.filterwarnings('ignore', category=UserWarning)

# Load dataset
df = pd.read_csv("wc1.csv")
X = df.drop('type', axis=1)
y = df['type']
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.33, random_st
print("Dataset wc1.csv loaded successfully.")
```

Dataset wc1.csv loaded successfully.

```
In [2]: # Importing Libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split, KFold
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import (
    confusion_matrix,
    roc_curve,
    auc,
    roc_auc_score,
    accuracy_score
)
```

```
In [3]: # Loading Dataset
df = pd.read_csv("wc1.csv")

print("First 5 rows of the dataset:")
print(df.head())
print("\nColumns in dataset:", df.columns.tolist())
```

First 5 rows of the dataset:

	inline_count	external_count	onclick_count	onload_count	onchange_count	\
0	21.0	23.0	1	131.0	0.0	
1	13.0	30.0	2	4.0	1.0	
2	0.0	3.0	1	0.0	0.0	
3	21.0	11.0	1	10.0	1.0	
4	10.0	5.0	1	0.0	0.0	

	avg_inline_script_block	avg_external_script_block	avg_onclick_count	\
0	0.0	662.062500	30737	
1	0.0	55.777778	2951	
2	0.0	207.000000	387	
3	0.0	104.800000	1505	
4	0.0	473.000000	4121	

	avg_onload_count	avg_onchange_count	avg_cyc_complexity	\
0	4	16	0.812500	
1	12	18	0.555556	
2	2	1	1.000000	
3	2	5	0.400000	
4	2	3	1.000000	

	library_code_count	type
0	13	alexa
1	10	alexa
2	1	phish
3	2	alexa
4	3	alexa

Columns in dataset: ['inline_count', 'external_count', 'onclick_count', 'onload_count', 'onchange_count', 'avg_inline_script_block', 'avg_external_script_block', 'avg_onclick_count', 'avg_onload_count', 'avg_onchange_count', 'avg_cyc_complexity', 'library_code_count', 'type']

```
In [4]: # Encoding Target Labels ('type' column)
# Converting string labels into numeric (0 = alexa, 1 = phish)
y = df['type'].map({'alexa': 0, 'phish': 1})

# Defining features
X = df.drop('type', axis=1)

print("Label distribution:\n", y.value_counts())
```

Label distribution:

```
type
0    17562
1     6670
Name: count, dtype: int64
```

```
In [5]: # Splitting Data (33% test)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.33, random_state=42, stratify=y
)
print(f"Training samples: {X_train.shape[0]}, Testing samples: {X_test.shape[0]}")
```

Training samples: 16235, Testing samples: 7997

```
In [6]: # Training Random Forest with 100 trees
rf_model = RandomForestClassifier(n_estimators=100, random_state=42)
rf_model.fit(X_train, y_train)

y_pred = rf_model.predict(X_test)
cm = confusion_matrix(y_test, y_pred)

print("Confusion Matrix (100 Trees):\n", cm)
print(f"Accuracy (100 Trees): {accuracy_score(y_test, y_pred):.4f}")
```

```
Confusion Matrix (100 Trees):
[[5493  303]
 [ 350 1851]]
Accuracy (100 Trees): 0.9183
```

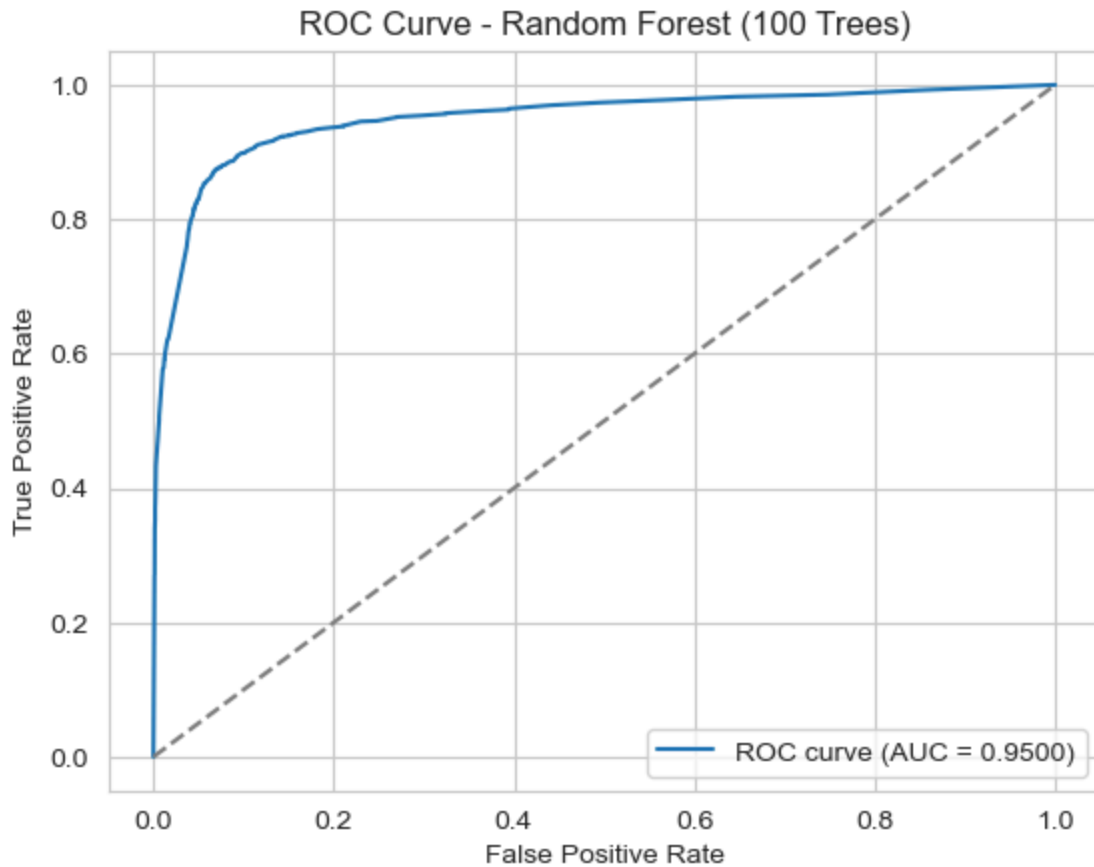
```
In [7]: # ROC and AUC (100 Trees)
y_pred_proba = rf_model.predict_proba(X_test)[:, 1]

fpr, tpr, thresholds = roc_curve(y_test, y_pred_proba)
roc_auc = auc(fpr, tpr)

print(f"AUC (100 Trees): {roc_auc:.4f}")

plt.figure()
plt.plot(fpr, tpr, label=f'ROC curve (AUC = {roc_auc:.4f})')
plt.plot([0, 1], [0, 1], linestyle='--', color='gray')
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('ROC Curve - Random Forest (100 Trees)')
plt.legend(loc="lower right")
plt.show()
```

```
AUC (100 Trees): 0.9500
```



```
In [8]: # Displaying Top 10 Important Features
feature_importances = pd.Series(
    rf_model.feature_importances_, index=X.columns
).sort_values(ascending=False)

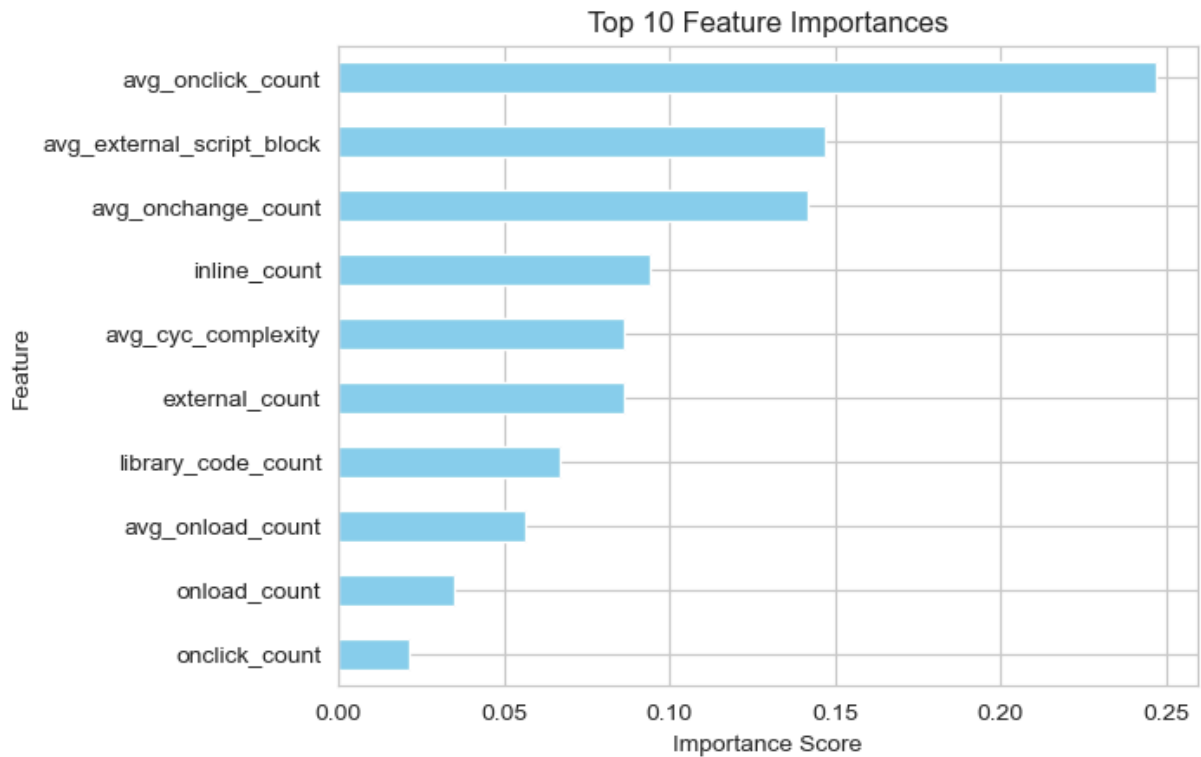
print("Top 10 Feature Importances:\n")
print(feature_importances.head(10))

feature_importances.head(10).plot(kind='barh', color='skyblue')
plt.title("Top 10 Feature Importances")
plt.xlabel("Importance Score")
plt.ylabel("Feature")
plt.gca().invert_yaxis()
plt.show()
```

Top 10 Feature Importances:

avg_onclick_count	0.247075
avg_external_script_block	0.147222
avg_onchange_count	0.141954
inline_count	0.094060
avg_cyc_complexity	0.086321
external_count	0.086080
library_code_count	0.067195
avg_onload_count	0.056709
onload_count	0.035078
onclick_count	0.021217

dtype: float64



```
In [9]: # Experimenting with More Trees
tree_counts = [200, 300, 400, 500, 600, 700, 800, 900]

for n_trees in tree_counts:
    rf_model = RandomForestClassifier(n_estimators=n_trees, random_state=42)
    rf_model.fit(X_train, y_train)

    y_pred = rf_model.predict(X_test)
    acc = accuracy_score(y_test, y_pred)
    cm = confusion_matrix(y_test, y_pred)

    print(f"\n==== Random Forest ({n_trees} Trees) =====")
    print(f"Accuracy: {acc:.4f}")
    print(f"Confusion Matrix:\n{cm}")
```

```
==== Random Forest (200 Trees) ====
Accuracy: 0.9185
Confusion Matrix:
[[5493  303]
 [ 349 1852]]
```

```
==== Random Forest (300 Trees) ====
Accuracy: 0.9190
Confusion Matrix:
[[5492  304]
 [ 344 1857]]
```

```
==== Random Forest (400 Trees) ====
Accuracy: 0.9187
Confusion Matrix:
[[5495  301]
 [ 349 1852]]
```

```
==== Random Forest (500 Trees) ====
Accuracy: 0.9186
Confusion Matrix:
[[5496  300]
 [ 351 1850]]
```

```
==== Random Forest (600 Trees) ====
Accuracy: 0.9185
Confusion Matrix:
[[5495  301]
 [ 351 1850]]
```

```
==== Random Forest (700 Trees) ====
Accuracy: 0.9186
Confusion Matrix:
[[5494  302]
 [ 349 1852]]
```

```
==== Random Forest (800 Trees) ====
Accuracy: 0.9183
Confusion Matrix:
[[5493  303]
 [ 350 1851]]
```

```
==== Random Forest (900 Trees) ====
Accuracy: 0.9190
Confusion Matrix:
[[5498  298]
 [ 350 1851]]
```

```
In [10]: # Experimenting with Different Tree Counts for Performance Comparison
tree_counts = [50, 100, 150, 200, 300]

for n_trees in tree_counts:
    rf_model = RandomForestClassifier(n_estimators=n_trees, random_state=42)
    rf_model.fit(X_train, y_train)

    y_pred = rf_model.predict(X_test)
```

```

acc = accuracy_score(y_test, y_pred)
cm = confusion_matrix(y_test, y_pred)

print(f"\n-- Model with {n_trees} Trees --")
print(f"Accuracy: {acc:.4f}")
print("Confusion Matrix:\n", cm)

```

-- Model with 50 Trees --

Accuracy: 0.9187

Confusion Matrix:

```

[[5489  307]
 [ 343 1858]]

```

-- Model with 100 Trees --

Accuracy: 0.9183

Confusion Matrix:

```

[[5493  303]
 [ 350 1851]]

```

-- Model with 150 Trees --

Accuracy: 0.9188

Confusion Matrix:

```

[[5496  300]
 [ 349 1852]]

```

-- Model with 200 Trees --

Accuracy: 0.9185

Confusion Matrix:

```

[[5493  303]
 [ 349 1852]]

```

-- Model with 300 Trees --

Accuracy: 0.9190

Confusion Matrix:

```

[[5492  304]
 [ 344 1857]]

```

```

In [11]: # Observing Stabilization at Optimal Tree Count (Example: 400 Trees)
         optimal_rf_model = RandomForestClassifier(n_estimators=400, random_state=42)
         optimal_rf_model.fit(X_train, y_train)

         opt_pred = optimal_rf_model.predict(X_test)
         opt_acc = accuracy_score(y_test, opt_pred)

         print(f"Optimal Model Accuracy (400 Trees): {opt_acc:.4f}")
         print("Optimal Confusion Matrix (400 Trees):\n", confusion_matrix(y_test, opt_pred))

```

Optimal Model Accuracy (400 Trees): 0.9187

Optimal Confusion Matrix (400 Trees):

```

[[5495  301]
 [ 349 1852]]

```

```

In [12]: # K-Fold Cross-Validation
         kfold = KFold(n_splits=10, shuffle=True, random_state=42)
         fold accuracies = []

         for train_idx, val_idx in kfold.split(X_train):

```

```

X_fold_train, X_fold_val = X_train.iloc[train_idx], X_train.iloc[val_idx]
y_fold_train, y_fold_val = y_train.iloc[train_idx], y_train.iloc[val_idx]

rf_model = RandomForestClassifier(n_estimators=400, random_state=42)
rf_model.fit(X_fold_train, y_fold_train)
y_fold_pred = rf_model.predict(X_fold_val)

acc = accuracy_score(y_fold_val, y_fold_pred)
fold_accuracies.append({'model': rf_model, 'accuracy': acc})

print(f"Fold Accuracy: {acc:.4f}")

print(f"\nAverage Cross-Validation Accuracy: {np.mean([f['accuracy'] for f in fold_

```

```

Fold Accuracy: 0.9236
Fold Accuracy: 0.9119
Fold Accuracy: 0.9187
Fold Accuracy: 0.9181
Fold Accuracy: 0.9224
Fold Accuracy: 0.9298
Fold Accuracy: 0.9119
Fold Accuracy: 0.9181
Fold Accuracy: 0.9316
Fold Accuracy: 0.9298

```

Average Cross-Validation Accuracy: 0.9216

```

In [13]: # Choosing Model with Highest Accuracy
best_rf_model = max(fold_accuracies, key=lambda x: x['accuracy'])['model']

best_pred = best_rf_model.predict(X_test)
best_acc = accuracy_score(y_test, best_pred)

print(f"Best Model Test Accuracy: {best_acc:.4f}")
print("Best Model Confusion Matrix:\n", confusion_matrix(y_test, best_pred))

```

```

Best Model Test Accuracy: 0.9145
Best Model Confusion Matrix:
[[5489  307]
 [ 377 1824]]

```

```

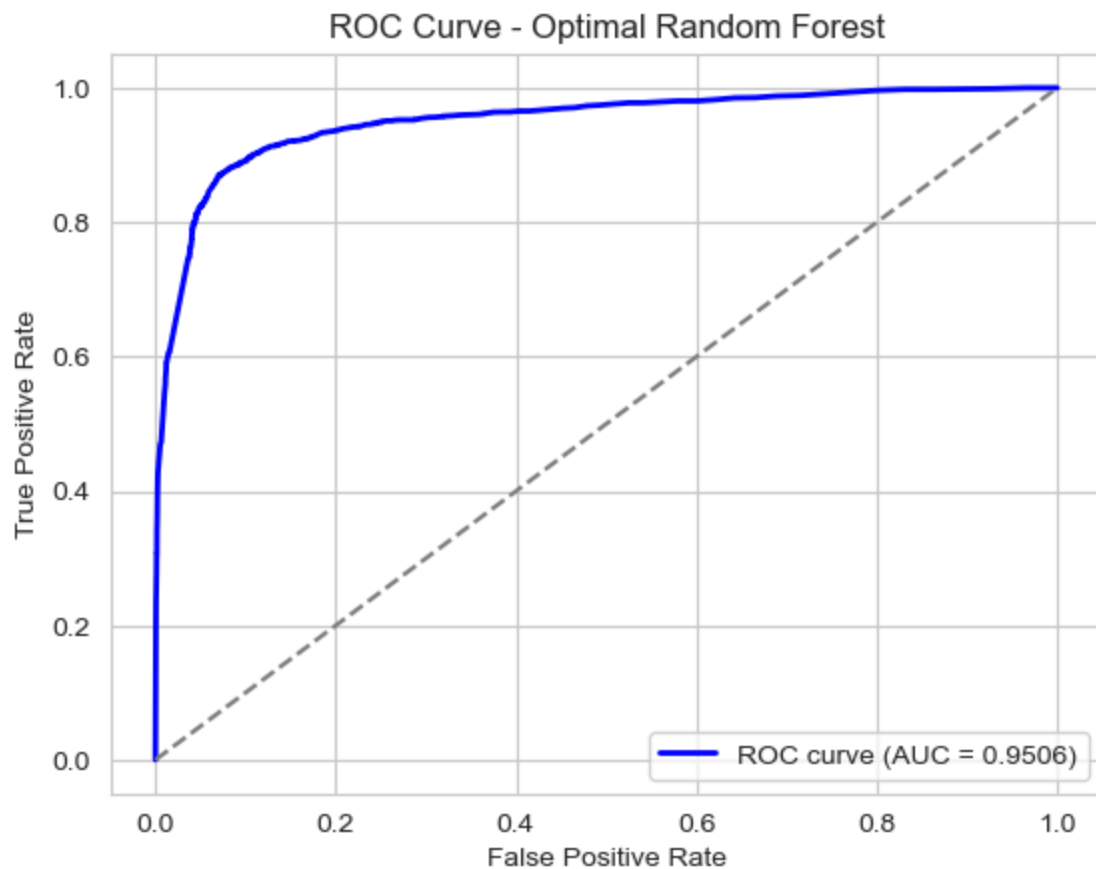
In [14]: # ROC and AUC for Best Model
best_proba = best_rf_model.predict_proba(X_test)[: , 1]
fpr, tpr, thresholds = roc_curve(y_test, best_proba)
roc_auc = roc_auc_score(y_test, best_proba)

print(f"ROC AUC Score for Best Model: {roc_auc:.4f}")

plt.figure()
plt.plot(fpr, tpr, color='blue', lw=2, label=f'ROC curve (AUC = {roc_auc:.4f})')
plt.plot([0, 1], [0, 1], linestyle='--', color='gray')
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('ROC Curve - Optimal Random Forest')
plt.legend(loc="lower right")
plt.show()

```


ROC AUC Score for Best Model: 0.9506



```
In [15]: # Project: Machine Learning Part II - Neural Network Classifier

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split, KFold, cross_val_score
from sklearn.preprocessing import StandardScaler
from sklearn.neural_network import MLPClassifier
from sklearn.metrics import confusion_matrix, roc_auc_score, accuracy_score, recall
import warnings

sns.set_style("whitegrid")
warnings.filterwarnings('ignore', category=UserWarning)

# --- Data Loading ---
try:
    df = pd.read_csv('wc1.csv')

    # Feature (X) and Target (y) identification
    X = df.drop('type', axis=1)
    y_raw = df['type']

    # Target Encoding: 'phish' (1, Positive Class), 'alex' (0, Negative Class)
    label_map = {'phish': 1, 'alex': 0}
    y = y_raw.map(label_map)
```

```

print("Data loaded and target encoded successfully.")
print(f"Dataset shape: {df.shape}")

except Exception as e:
    print(f"ERROR: Could not load data. Ensure 'wc1.csv' is accessible. Error: {e}")

```

Data loaded and target encoded successfully.

Dataset shape: (24232, 13)

```

In [16]: # Data Splitting (Requirement: 33% Testing)
TEST_SIZE = 0.33
RANDOM_STATE = 1

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=TEST_SIZE, random_state=RANDOM_STATE, stratify=y
)

print(f"--- Data Split (Test Size: {TEST_SIZE*100:.0f}%) ---")
print(f"Training Samples: {X_train.shape[0]}, Testing Samples: {X_test.shape[0]}")

# 4. Feature Scaling (MANDATORY FOR MLPClassifier)
# This standardizes the features (mean=0, variance=1)
scaler = StandardScaler()

# Fit scaler only on the training data and transform both sets
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

print("\nFeatures have been successfully scaled using StandardScaler.")

```

--- Data Split (Test Size: 33%) ---

Training Samples: 16235, Testing Samples: 7997

Features have been successfully scaled using StandardScaler.

```

In [17]: # --- PART 1: Initial Neural Network Evaluation (max_iter=200) ---
MAX_ITER_INITIAL = 200

print(f"\n--- Initial Model Run (max_iter={MAX_ITER_INITIAL}) ---")

# Use default hidden layers (single layer of size 100) and turn verbose on to capture
mlp_initial = MLPClassifier(
    random_state=RANDOM_STATE,
    max_iter=MAX_ITER_INITIAL,
    verbose=True
)

# Train the model on the scaled data
mlp_initial.fit(X_train_scaled, y_train)

# Output convergence status
print(f"\nModel Stopped at max_iter: {mlp_initial.n_iter_ == MAX_ITER_INITIAL}")

# Initial Confusion Matrix and Metrics
y_pred_initial = mlp_initial.predict(X_test_scaled)
cm_initial = confusion_matrix(y_test, y_pred_initial)

```

```
print(f"\nConfusion Matrix (max_iter={MAX_ITER_INITIAL}):\n{cm_initial}")
```

--- Initial Model Run (max_iter=200) ---

Iteration 1, loss = 0.54014102
Iteration 2, loss = 0.44229841
Iteration 3, loss = 0.41944194
Iteration 4, loss = 0.40397137
Iteration 5, loss = 0.39318570
Iteration 6, loss = 0.38631914
Iteration 7, loss = 0.38174358
Iteration 8, loss = 0.37783702
Iteration 9, loss = 0.37504605
Iteration 10, loss = 0.37292540
Iteration 11, loss = 0.37043158
Iteration 12, loss = 0.36893428
Iteration 13, loss = 0.36739732
Iteration 14, loss = 0.36585197
Iteration 15, loss = 0.36439189
Iteration 16, loss = 0.36351797
Iteration 17, loss = 0.36264598
Iteration 18, loss = 0.36126676
Iteration 19, loss = 0.36037800
Iteration 20, loss = 0.36057134
Iteration 21, loss = 0.35913088
Iteration 22, loss = 0.35861333
Iteration 23, loss = 0.35835641
Iteration 24, loss = 0.35691685
Iteration 25, loss = 0.35643017
Iteration 26, loss = 0.35650101
Iteration 27, loss = 0.35514364
Iteration 28, loss = 0.35422364
Iteration 29, loss = 0.35323950
Iteration 30, loss = 0.35312690
Iteration 31, loss = 0.35292197
Iteration 32, loss = 0.35223004
Iteration 33, loss = 0.35182727
Iteration 34, loss = 0.35182432
Iteration 35, loss = 0.35102778
Iteration 36, loss = 0.35042747
Iteration 37, loss = 0.35046233
Iteration 38, loss = 0.35030599
Iteration 39, loss = 0.34959774
Iteration 40, loss = 0.34872894
Iteration 41, loss = 0.34940286
Iteration 42, loss = 0.34819011
Iteration 43, loss = 0.34760297
Iteration 44, loss = 0.34741333
Iteration 45, loss = 0.34793381
Iteration 46, loss = 0.34635616
Iteration 47, loss = 0.34621285
Iteration 48, loss = 0.34557980
Iteration 49, loss = 0.34632145
Iteration 50, loss = 0.34567916
Iteration 51, loss = 0.34517067
Iteration 52, loss = 0.34446034
Iteration 53, loss = 0.34420655
Iteration 54, loss = 0.34480991
Iteration 55, loss = 0.34421471

Iteration 56, loss = 0.34391050
Iteration 57, loss = 0.34325590
Iteration 58, loss = 0.34371267
Iteration 59, loss = 0.34258771
Iteration 60, loss = 0.34254080
Iteration 61, loss = 0.34209941
Iteration 62, loss = 0.34244262
Iteration 63, loss = 0.34178473
Iteration 64, loss = 0.34157145
Iteration 65, loss = 0.34100253
Iteration 66, loss = 0.34155952
Iteration 67, loss = 0.34059019
Iteration 68, loss = 0.34178988
Iteration 69, loss = 0.34033747
Iteration 70, loss = 0.34046979
Iteration 71, loss = 0.34022757
Iteration 72, loss = 0.33993476
Iteration 73, loss = 0.34007928
Iteration 74, loss = 0.33911291
Iteration 75, loss = 0.33905511
Iteration 76, loss = 0.33799894
Iteration 77, loss = 0.33872998
Iteration 78, loss = 0.33806504
Iteration 79, loss = 0.33819896
Iteration 80, loss = 0.33825661
Iteration 81, loss = 0.33779483
Iteration 82, loss = 0.33788880
Iteration 83, loss = 0.33739528
Iteration 84, loss = 0.33632800
Iteration 85, loss = 0.33658094
Iteration 86, loss = 0.33614462
Iteration 87, loss = 0.33648159
Iteration 88, loss = 0.33587146
Iteration 89, loss = 0.33541357
Iteration 90, loss = 0.33488110
Iteration 91, loss = 0.33544730
Iteration 92, loss = 0.33509977
Iteration 93, loss = 0.33547878
Iteration 94, loss = 0.33571581
Iteration 95, loss = 0.33448815
Iteration 96, loss = 0.33452103
Iteration 97, loss = 0.33526993
Iteration 98, loss = 0.33416022
Iteration 99, loss = 0.33408125
Iteration 100, loss = 0.33338080
Iteration 101, loss = 0.33316183
Iteration 102, loss = 0.33436824
Iteration 103, loss = 0.33343197
Iteration 104, loss = 0.33322417
Iteration 105, loss = 0.33420853
Iteration 106, loss = 0.33208332
Iteration 107, loss = 0.33305404
Iteration 108, loss = 0.33231594
Iteration 109, loss = 0.33264841
Iteration 110, loss = 0.33168472
Iteration 111, loss = 0.33259266

Iteration 112, loss = 0.33136955
Iteration 113, loss = 0.33189263
Iteration 114, loss = 0.33265830
Iteration 115, loss = 0.33185259
Iteration 116, loss = 0.33125379
Iteration 117, loss = 0.33025415
Iteration 118, loss = 0.33027444
Iteration 119, loss = 0.32968560
Iteration 120, loss = 0.33107208
Iteration 121, loss = 0.33105056
Iteration 122, loss = 0.32995032
Iteration 123, loss = 0.32979800
Iteration 124, loss = 0.32931030
Iteration 125, loss = 0.32927219
Iteration 126, loss = 0.32911238
Iteration 127, loss = 0.32962150
Iteration 128, loss = 0.32885306
Iteration 129, loss = 0.32908705
Iteration 130, loss = 0.32815059
Iteration 131, loss = 0.32842204
Iteration 132, loss = 0.32832759
Iteration 133, loss = 0.32819715
Iteration 134, loss = 0.32796996
Iteration 135, loss = 0.32767432
Iteration 136, loss = 0.32824579
Iteration 137, loss = 0.32696415
Iteration 138, loss = 0.32790320
Iteration 139, loss = 0.32809835
Iteration 140, loss = 0.32801587
Iteration 141, loss = 0.32662280
Iteration 142, loss = 0.32662923
Iteration 143, loss = 0.32702458
Iteration 144, loss = 0.32766964
Iteration 145, loss = 0.32691243
Iteration 146, loss = 0.32549886
Iteration 147, loss = 0.32805826
Iteration 148, loss = 0.32631299
Iteration 149, loss = 0.32595551
Iteration 150, loss = 0.32542596
Iteration 151, loss = 0.32630376
Iteration 152, loss = 0.32542007
Iteration 153, loss = 0.32536583
Iteration 154, loss = 0.32503497
Iteration 155, loss = 0.32501266
Iteration 156, loss = 0.32452436
Iteration 157, loss = 0.32481752
Iteration 158, loss = 0.32511376
Iteration 159, loss = 0.32540215
Iteration 160, loss = 0.32469686
Iteration 161, loss = 0.32415456
Iteration 162, loss = 0.32369069
Iteration 163, loss = 0.32362538
Iteration 164, loss = 0.32458028
Iteration 165, loss = 0.32307677
Iteration 166, loss = 0.32325495
Iteration 167, loss = 0.32379637

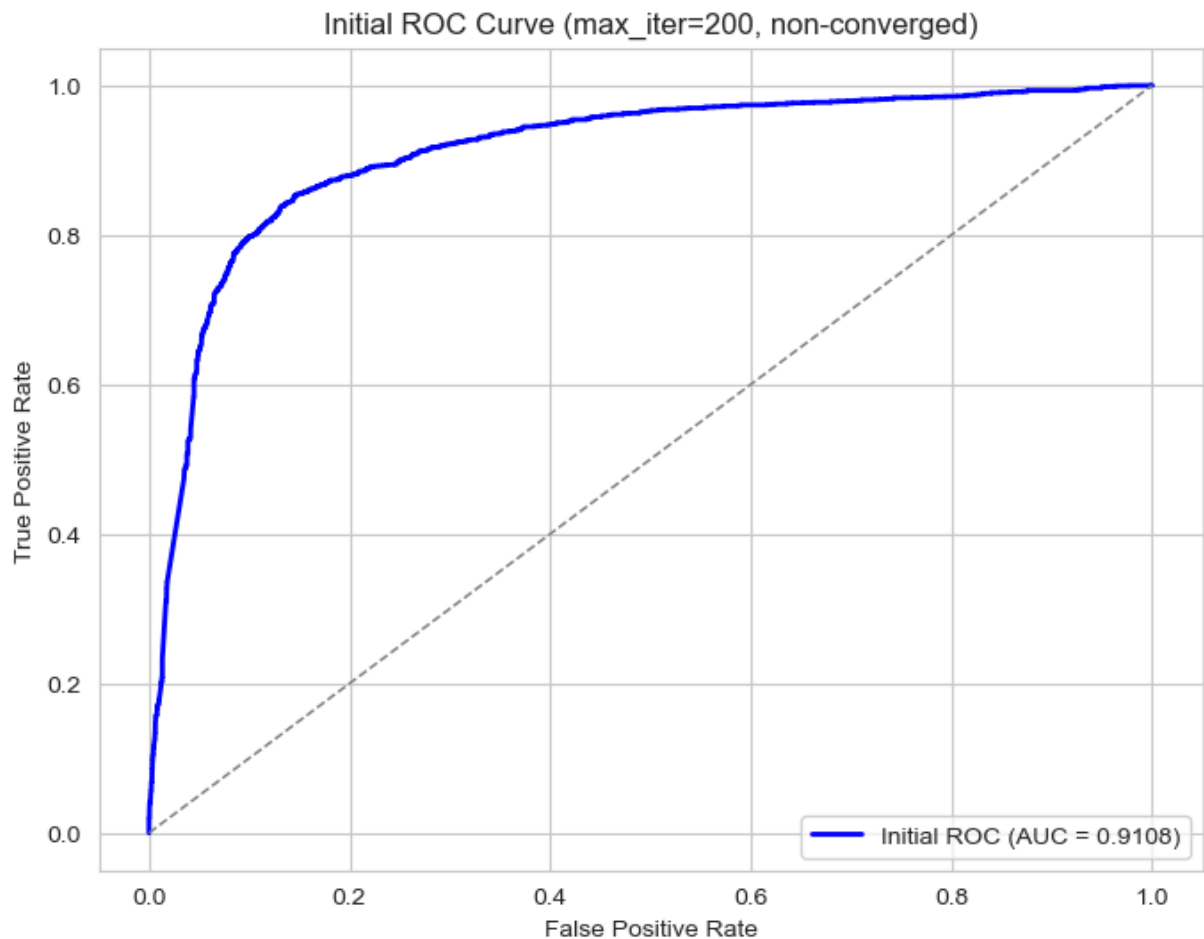
```
Iteration 168, loss = 0.32389733
Iteration 169, loss = 0.32323918
Iteration 170, loss = 0.32322568
Iteration 171, loss = 0.32324684
Iteration 172, loss = 0.32347233
Iteration 173, loss = 0.32238255
Iteration 174, loss = 0.32297802
Iteration 175, loss = 0.32200287
Iteration 176, loss = 0.32204676
Iteration 177, loss = 0.32187177
Iteration 178, loss = 0.32261897
Iteration 179, loss = 0.32194959
Iteration 180, loss = 0.32177341
Iteration 181, loss = 0.32121607
Iteration 182, loss = 0.32152971
Iteration 183, loss = 0.32166800
Iteration 184, loss = 0.32032122
Iteration 185, loss = 0.32054270
Iteration 186, loss = 0.32096113
Iteration 187, loss = 0.32011572
Iteration 188, loss = 0.32026499
Iteration 189, loss = 0.32052406
Iteration 190, loss = 0.32070701
Iteration 191, loss = 0.31956781
Iteration 192, loss = 0.32053635
Iteration 193, loss = 0.32030261
Iteration 194, loss = 0.31893517
Iteration 195, loss = 0.31978455
Iteration 196, loss = 0.31993009
Iteration 197, loss = 0.32038037
Iteration 198, loss = 0.31920786
Iteration 199, loss = 0.31868560
Iteration 200, loss = 0.31992198
```

Model Stopped at max_iter: True

```
Confusion Matrix (max_iter=200):
[[5365  431]
 [ 576 1625]]
```

```
In [18]: # Plotting ROC curve for the initial (non-converged) model
y_prob_initial = mlp_initial.predict_proba(X_test_scaled)[: , 1]
fpr_initial, tpr_initial, _ = roc_curve(y_test, y_prob_initial)
auc_initial = roc_auc_score(y_test, y_prob_initial)

plt.figure(figsize=(8, 6))
plt.plot(fpr_initial, tpr_initial, color='blue', lw=2, label=f'Initial ROC (AUC = {
plt.plot([0, 1], [0, 1], color='gray', lw=1, linestyle='--')
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('Initial ROC Curve (max_iter=200, non-converged)')
plt.legend(loc="lower right")
plt.grid(True)
plt.show()
```



```
In [19]: # --- PART 2: Experimentation to Achieve Convergence ---

iteration_settings = [200, 400, 600, 800, 1000]
results = []
MAX_ITER_FINAL = 0

print("\n--- Experimenting with Increased max_iter ---")

for max_i in iteration_settings:
    mlp = MLPClassifier(random_state=RANDOM_STATE, max_iter=max_i)

    # Train model
    mlp.fit(X_train_scaled, y_train)

    y_pred = mlp.predict(X_test_scaled)
    y_prob = mlp.predict_proba(X_test_scaled)[:, 1]

    current_auc = roc_auc_score(y_test, y_prob)
    current_acc = accuracy_score(y_test, y_pred)

    converged = mlp.n_iter_ < max_i

    results.append({
        'max_iter': max_i,
        'AUC': current_auc,
        'Accuracy': current_acc,
```



```

        'Converged': converged
    })

    print(f"max_iter={max_i:4d}: AUC={current_auc:.4f}, Accuracy={current_acc:.4f},

    if converged and MAX_ITER_FINAL == 0:
        MAX_ITER_FINAL = max_i

# Set final max_iter based on first converged point, or fallback to highest tested
    if MAX_ITER_FINAL == 0:
        MAX_ITER_FINAL = max_i
        print(f"\nModel did not strictly converge up to max_iter={MAX_ITER_FINAL}. Usin
    else:
        print(f"\nFinal Convergence Setting Selected: max_iter={MAX_ITER_FINAL}")

```

```

--- Experimenting with Increased max_iter ---
max_iter= 200: AUC=0.9108, Accuracy=0.8741, Converged=False
max_iter= 400: AUC=0.9158, Accuracy=0.8796, Converged=False
max_iter= 600: AUC=0.9152, Accuracy=0.8798, Converged=True
max_iter= 800: AUC=0.9152, Accuracy=0.8798, Converged=True
max_iter=1000: AUC=0.9152, Accuracy=0.8798, Converged=True

```

Final Convergence Setting Selected: max_iter=600

```

In [20]: # Final Model Training and Full Evaluation (max_iter = MAX_ITER_FINAL)

print(f"\n--- Final Model Training (max_iter={MAX_ITER_FINAL}) ---")

mlp_final = MLPClassifier(random_state=RANDOM_STATE, max_iter=MAX_ITER_FINAL)
mlp_final.fit(X_train_scaled, y_train)

# Metrics Calculation
y_pred_final = mlp_final.predict(X_test_scaled)
y_prob_final = mlp_final.predict_proba(X_test_scaled)[: , 1]

cm_final = confusion_matrix(y_test, y_pred_final)
tn, fp, fn, tp = cm_final.ravel()

final_accuracy = accuracy_score(y_test, y_pred_final)
final_recall = recall_score(y_test, y_pred_final)
final_precision = precision_score(y_test, y_pred_final)
final_fpr = fp / (fp + tn)
final_auc = roc_auc_score(y_test, y_prob_final)

print(f"\nFinal Test Accuracy: {final_accuracy:.4f}")
print(f"Final Recall (TP Rate): {final_recall:.4f}")
print(f"Final FPR (FP Rate): {final_fpr:.4f}")
print(f"Final AUC Score: {final_auc:.4f}")

# K-FOLD CROSS-VALIDATION (5-Fold)
print("\n--- K-Fold Cross-Validation (k=5) ---")
# Use the full scaled dataset (X_scaled) for CV
X_full_scaled = scaler.fit_transform(X)
cv_scores = cross_val_score(mlp_final, X_full_scaled, y, cv=5, scoring='accuracy')

```

```
print(f"CV Accuracy Scores: {cv_scores}")
print(f"Average CV Accuracy: {cv_scores.mean():.4f} (Standard Deviation: {cv_scores
```

--- Final Model Training (max_iter=600) ---

Final Test Accuracy: 0.8798
 Final Recall (TP Rate): 0.7715
 Final FPR (FP Rate): 0.0790
 Final AUC Score: 0.9152

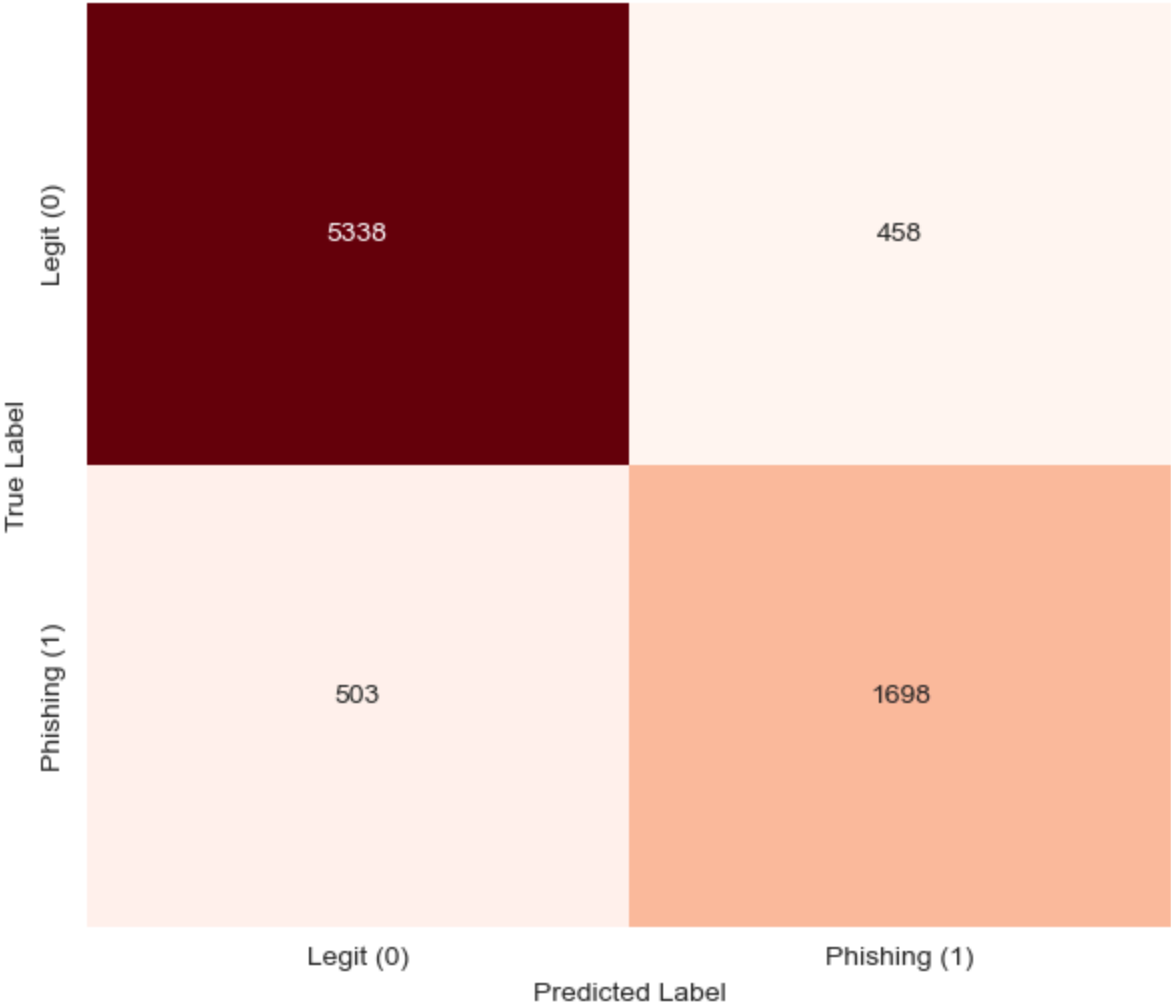
--- K-Fold Cross-Validation (k=5) ---

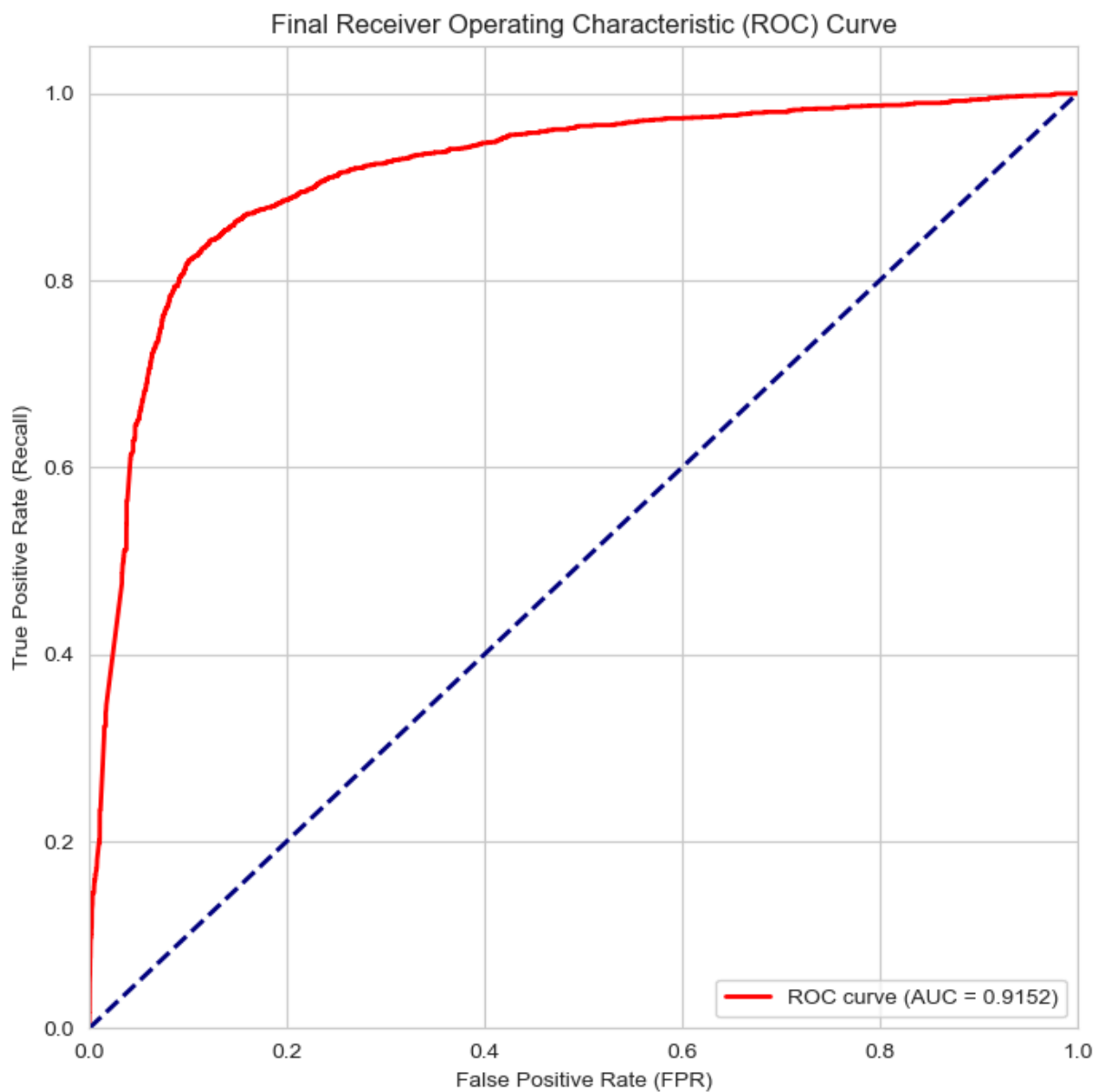
CV Accuracy Scores: [0.87724366 0.87414896 0.87226579 0.8708213 0.88031366]
 Average CV Accuracy: 0.8750 (Standard Deviation: 0.0034)

```
In [21]: # Plotting the Final Confusion Matrix (Required Visual)
plt.figure(figsize=(7, 6))
sns.heatmap(cm_final, annot=True, fmt='d', cmap='Reds', cbar=False,
            xticklabels=['Legit (0)', 'Phishing (1)'],
            yticklabels=['Legit (0)', 'Phishing (1)'])
plt.title(f'Final Confusion Matrix (max_iter={MAX_ITER_FINAL})', fontsize=14)
plt.ylabel('True Label')
plt.xlabel('Predicted Label')
plt.show()

# Plotting the Final ROC Curve (Required Visual)
fpr_final, tpr_final, _ = roc_curve(y_test, y_prob_final)
plt.figure(figsize=(8, 8))
plt.plot(fpr_final, tpr_final, color='red', lw=2, label=f'ROC curve (AUC = {final_a
plt.plot([0, 1], [0, 1], color='navy', lw=2, linestyle='--')
plt.xlim([0.0, 1.0])
plt.ylim([0.0, 1.05])
plt.xlabel('False Positive Rate (FPR)')
plt.ylabel('True Positive Rate (Recall)')
plt.title('Final Receiver Operating Characteristic (ROC) Curve')
plt.legend(loc="lower right")
plt.grid(True)
plt.show()
```

Final Confusion Matrix (max_iter=600)





In []: